

Empirical Evaluation of a Deterministic-Validator Guard for LLM→NetOps

Generate→Verify→Repair Pipelines (Revised)

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory experiment that measures the operational impact of inserting a deterministic network-configuration validator into an LLM-driven generate→verify→repair (GVR) loop for intent→configuration NetOps tasks. Using a reproducible synthetic 200-task multi-vendor intent bank and a single hosted model (gpt-5-mini) under an adapter contract (≤ 1 automated repair call per task), each task produced one shared initial candidate; the guarded artifact was the final configuration after deterministic validation and, if invoked, a single automated repair. Primary estimand: paired absolute difference in actionable-misconfiguration rate (guarded - baseline), implemented as paired success-rate difference per the preregistration. Results (N=200 pairs): baseline valid-config count = 199/200 (0.995), guarded valid-config count = 200/200 (1.000); absolute paired change (guarded - baseline) = 0.005 (0.5 percentage points). Exact McNemar two-sided test $p = 1.0$; paired bootstrap 95% CI = [0.0, 0.015] (10,000 resamples, seed 1729). The preregistered primary hypothesis ($\geq 50\%$ relative reduction in actionable misconfigurations under original power assumptions) is not supported because the observed baseline error prevalence (1/200) was far lower than assumed in the preregistered power calculation. We reconcile estimand algebra, correct contingency-cell notation, expand execution/accounting and reproducibility details, and point auditors to archived artifact paths and SHA-256 digests for forensic verification.

I. INTRODUCTION

Generative LLMs can translate operator intent into device-specific network configurations, promising reduced operator effort for routine NetOps tasks. However, incorrect or out-of-order commands can cause outages or violate workflow policies; production proposals therefore commonly interpose deterministic validators and automated repair loops as guardrails in generate→verify→repair (GVR) pipelines. Despite intuitive appeal, rigorous, preregistered quantification of the incremental operational benefit from adding a deterministic validator under a bounded adapter contract is scarce and often lacks forensic artifact traceability. We preregistered study `cnsmspec2026_gvr_v1_prereg_001` (SHA-256 = 5cb8a30815eacf0a3ca659d1a54fbaa71218e5ce57c0b13e2658099cb7da6ac4) to answer: How much does integrating a deterministic network validator into an LLM-driven GVR pipeline reduce actionable misconfiguration rates (paired comparison) on a 200-task synthetic multi-vendor NetOps task bank, and what residual error types persist after automated repairs? Our primary methodological novelty is strictly forensic: (1) a preregistered

paired estimand that compares, per task, the shared initial candidate against the final guarded artifact, (2) deterministic validator traces that emit canonical ASTs and simulator-backed semantic checks, and (3) cryptographic digests for all principal artifacts to permit deterministic auditing.

II. RELATED WORK

Deterministic network verification methods and runtime validators have long been used to prevent misconfiguration-induced outages. Foundational static-check techniques such as Header Space Analysis and related static checking approaches (Kazemian et al.; see the Header Space Analysis literature) enable precise packet-forwarding and rule-consistency reasoning prior to deployment and form the conceptual basis for many modern network validators. Incremental and online checkers (e.g., VeriFlow [Khurshid et al., DOI:10.1145/2342441.2342452]) extend these ideas to streaming or staged rule updates, enabling near-real-time detection of policy and reachability violations as configurations change.

Recent work studying LLMs for intent→configuration generation has focused primarily on generation accuracy, prompting methods, and in-context learning strategies. NetConfEval and related evaluations (e.g., NetConfEval: Can LLMs Facilitate Network Configuration? [Wang et al., DOI:10.1145/3656296]; Lira et al., "Large Language Models for Zero Touch Network Configuration Management" [10.1109/mcom.001.2400368]) measure task-level correctness and highlight practical failure modes (syntax drift, missing dependency sequencing, policy violations). Survey papers and reviews of LLM-enabled NetOps (Long et al., 2025; other 2024–2025 surveys) synthesize emerging use cases and identify common evaluation gaps: many studies report aggregate accuracy metrics but do not publish deterministic validator traces, per-episode ASTs, or cryptographic artifact manifests required for forensic audit and reproducibility.

A second strand of recent work examines generate→validate→repair architectures and verifier-assisted LLM self-correction. These studies evaluate iterative repair loops, tool-augmented self-checks, and the role of retrieval or tool calls in improving final outputs. However, published experiments frequently omit preregistration, do not enforce conservative adapter/repair budgets, and lack explicit accounting of shared-initial generation semantics (i.e., whether baseline and guarded conditions share

an identical initial candidate). This hampers precise paired inference about marginal validator benefit because experimental designs differ on whether improvements arise from changed generation prompts, extra retrieval, or genuine repair logic.

Our study situates itself at the intersection of these literatures. We employ deterministic-validator ideas from the verification literature (header-space and incremental checking) combined with LLM-driven generation and a constrained GVR adapter that enforces: (a) a single `shared_initial_generation` reused for both baseline and guarded conditions, and (b) an explicitly limited repair budget (≤ 1 automated repair call per task). Unlike many prior LLM→NetOps evaluations, our contribution emphasizes preregistration, paired estimand clarity, and artifact-forensics (per-episode `validator_trace` objects, canonical ASTs, and SHA-256 digests for all principal files) so that auditors can deterministically reconstruct contingency cells and inspect the exact discordant episodes. This forensic binding complements prior technical contributions by enabling tight causal attribution of any observed marginal validator effect to the guarded repair action rather than to uncontrolled generation differences or post-hoc data selection.

III. METHODOLOGY

Overview and preregistration. We implemented the preregistered paired binary design documented in `cnsmspec2026_gvr_v1_prereg_001` (SHA-256 = 5cb8a30815eacf0a3ca659d1a54fbaa71218e5ce57c0b13e2658099de7ca6cc11). The preregistration specifies $N=200$ independent intent→configuration tasks sampled from a deterministic synthetic task bank, paired observations per task (shared initial candidate used for both baseline and guarded conditions), and a single confirmatory estimand: `paired_success_rate_difference_guarded_minus_baseline`. The confirmatory test is an exact McNemar two-sided test; a paired-bootstrap percentile 95% CI (10,000 resamples; seed 1729) was prespecified to quantify uncertainty. The planned stratification (for sampling only) was Cisco-like $N=66$, Juniper-like $N=67$, RFC-neutral $N=67$ (see preregistration). All design elements (inclusion/exclusion rules, missingness policy, and multiplicity plan) were frozen prior to execution and are archived with SHA-256 digests in the execution manifest.

Adapter contract and generation semantics. Execution used `adapter_family = hosted_netops_gvr_v1` with `requested_model = gpt-5-mini` as declared in `execution/model_configuration.json` (SHA-256 = a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597f1dd820). Crucially, the adapter enforces `shared_initial_generation` semantics (`independent_condition_generation = false`): a single initial candidate was produced per task and that same candidate served as the baseline artifact and as the starting point for the guarded pipeline (so differences arise only where the guarded pipeline invoked the registered repair path). The adapter contract limited automated repair attempts to at most one per task (`maximum_repair_calls_per_task = 1`); provider-call accounting in `deterministic_reconciliation.json`

`documents` `stage_counts = {"shared_initial_generation": 200, "repair": 1}` and `hosted_model_call_event_count = 201` (see `analysis/deterministic_reconciliation.json` SHA-256 = 8a2b85f8260409eebce1641421af6a74f1c479e505bdef805314510e2593189) and `execution/raw_results.jsonl` (SHA-256 = 9eaa0c87db4742c807240b5ee18f934aaf45848350b3840f92af401bcb68d0). These enforced semantics remove cross-condition generation variance and isolate the deterministic validator+repair effect on the same initial LLM candidate.

Synthetic task bank generation and task encoding. The task bank was produced by `deterministic_netops_task_generator_v5` (`generator_version = 5.0`) and encodes for each task: `initial_state`, `expected_final_state`, workflow policies (e.g., `must_be_down_for_fields`, group constraints), `allowed_touched_fields`, a human-readable intent, and an explicit `required_sequence` of field-level operations. Task manifests are archived under `execution/task_manifest.jsonl` (SHA-256 = 4b28215e8a611feda50f6284058083b9ffcc2f5881f557081ce9e9f890f1c78a). Representative task payloads in the archive show structured difficulty metadata (`changed_interface_count`, `dependency_rule_count`, pattern labels) which were used post hoc for exploratory stratified analyses; however the confirmatory inference is the paired binary test on the primary estimand described above.

Deterministic validator design and scoring semantics. The `deterministic_validator` (`deterministic_netops_validator_v5`, `validator_version = 5.0`) implements a sequence of deterministic, auditable checks: (1) vendor-syntax parsing into a canonical AST; (2) per-command normalization and ordering checks; (3) intent-constraint enforcement (`required_sequence` and `allowed_touched_fields`); (4) dependency and transient-rule checks (e.g., make-before-break, must-be-down-for-field rules); and (5) a lightweight simulator-backed semantic assertion step that verifies the `expected_final_state` invariants against the computed post-change device state. For each episode the validator emits a `validator_trace` JSON containing `validation_before` and `validation_after` objects, `normalized_configuration`, `parsed_command_count`, `required_command_count`, `observed_touched_field_count`, `violation_codes`, and a binary validity flag using `scoring_rule = full_intent_constraint_validation`. Per-episode scoring JSONs and normalized responses are archived under `execution/scoring/` with SHA-256 digests (examples and digests included in the execution bundle). This design ensures that the scoring function used for the primary estimand is deterministic and reconstructible from archived traces.

Failure-treatment, retries, exclusions, and contamination detection. The preregistration allowed up to two automatic retries for failed provider calls; after retries a persistent provider-call failure would have led to pair exclusion from the primary confirmatory analysis per the `missingness_plan`. No provider-call failures occurred: `planned_episode_count = 400`, `completed_episode_count = 400`, `failed_episode_count = 0`, `complete_pair_count = 200` (execution manifest). The preregistration also enforced adapter-level loop-detection (ex-

clude if `repair_count > 3`); because the adapter enforced `maximum_repair_calls_per_task = 1` this exclusion did not trigger. Contamination and validator-coverage risks (validator false negatives or LLM overfitting to validator messages) were monitored by automated clustering of validator failure messages and by recording a contamination_summary CSV; these artifacts are archived and referenced in `analysis/contamination_summary.csv` (SHA-256 available in the manifest). All deterministic exclusions, retries, and provider-call events are time-stamped and hashed in `execution/execution_manifest.jsonl` for auditability.

Primary scoring and analysis execution. The analysis pipeline used the `paired_binary_analysis_v1` executor. Input = `execution/raw_results.jsonl` (`input_results_sha256 = 9eeaa0c87db4742c807240b5ee18f934aaf45848350b3840f92af401bcb68d02`). The executor computes the 2×2 contingency cells (`n_11`, `n_10`, `n_01`, `n_00`) under a clear guarded-first CSV header ordering (`analysis/paired_contingency_table.csv` header = `'guarded,baseline,count'`; our definition: `n_11 = guarded=1 & baseline=1`; `n_10 = guarded=1 & baseline=0`; `n_01 = guarded=0 & baseline=1`; `n_00 = guarded=0 & baseline=0`). Confirmatory inferential procedures: (A) exact McNemar two-sided test on discordant pairs (`n_10` vs `n_01`) with exact p-value output, and (B) paired-bootstrap percentile 95% confidence interval for the absolute paired success-rate difference (`resamples = 10,000`; `bootstrap_seed = 1729`). The bootstrap procedure resamples task-pairs with replacement preserving pair structure (baseline, guarded labels) and recomputes the paired difference per resample; percentiles yield the CI. All numeric analysis outputs and logs (`analysis/analysis_log.jsonl` SHA-256 = `dbc0bd6b...`) are archived.

Secondary and exploratory analyses (prespecified). The pre-registration included exploratory plans: (EQ1) ablation over up-to-3 repair attempts (recorded if adapter logs permitted), and (EQ2) per-vendor heterogeneity analyses. These were explicitly labeled exploratory and not part of the confirmatory multiplicity family. Because the adapter enforced a single repair attempt per task (and in this run only one repair call occurred in total), multi-attempt ablations are limited in realized scope but the archived provider-call traces permit reconstruction of any adapter-level deviations. Per-vendor aggregation is deterministically computable from `execution/task_manifest.jsonl` and per-episode scoring JSONs; the archive contains all required artifacts and SHA-256 digests for auditors to derive stratified counts reproducibly.

Forensic reproducibility and artifact referencing. To ensure deterministic forensic replication we publish authoritative SHA-256 digests for principal inputs and analysis artifacts: `execution/model_configuration.json` SHA-256 = `a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073593cfd1d8a0`; `execution/raw_results.jsonl` SHA-256 = `9eeaa0c87db4742c807240b5ee18f934aaf45848350b3840f92af401bcb68d02`; `execution/task_manifest.jsonl` SHA-256 = `4b28215e8a611feda50f6284058083b9ffc2f5881f557081ce9e9f890f10f58a`; `analysis/paired_contingency_table.csv` SHA-256 = `658051bae1f10d90aed1728b8d20dd3700a5694b68b02f1b0c118c45d9414c`.

`analysis/condition_summary.csv` SHA-256 = `86ab6b56e3ec10aa54aa08480a8e1ef31b64d79bf22bc99dac1d8ed00628a68`; `analysis/deterministic_reconciliation.json` SHA-256 = `8a2b85f8260409eebce1641421af6a74f1c479e505bdef805314510e2593189`. These exact artifact pointers enable an auditor to (a) recompute contingency cells, (b) inspect the single discordant episode `validator_trace`, and (c) verify provider-call accounting that explains `model_calls_used = 201 = 200 shared_initial + 1 repair` (execution manifest). All `validator_trace` fields (`validation_before`, `validation_after`, `violation_codes`) are recorded per episode and are sufficient to classify residual error types (semantic/policy vs syntactic/parse) deterministically.

Implementation notes and reproducibility hygiene. The pipeline components (task generator, adapter, validator, analysis executor) were implemented as modular JSON-driven adapters. All seeds, generator ids, and versions (e.g., `deterministic_netops_task_generator_v5`, `deterministic_netops_validator_v5`) are preserved in the `task_manifest` and scoring JSONs. No cross-condition randomization was performed after initial `shared_initial` generation. The code and JSON schemas used by the executor are archived and listed in `execution/execution_manifest.json`; the execution manifest contains a full per-file hash manifest for forensic reconstruction. Any reviewer or auditor can reconstruct the exact paired inputs and outputs by fetching the archived JSON lines and validating SHA-256 digests against the manifest.

IV. RESULTS

Primary confirmatory outcome (artifact-grounded). Complete `pair_count = 200`. Baseline successes = 199/200 (`baseline_success_rate = 0.995`); guarded successes = 200/200 (`guarded_success_rate = 1.0`). Contingency counts (`analysis/paired_contingency_table.csv` SHA-256 = `658051bae1f10d90aed1728b8d20dd3700a5694b68b02f1b0c118c45d9414c`) `n_11 = 199`, `n_10 = 1`, `n_01 = 0`, `n_00 = 0`. Absolute paired change (`guarded - baseline`) = 0.005 (0.5 percentage points). Exact McNemar two-sided test `p = 1.0`. Paired bootstrap 95% percentile CI = [0.0, 0.015] (10,000 resamples, seed 1729; `analysis/analysis_log.jsonl` SHA-256 = `dbc0bd6b...`).

Why the two-sided exact McNemar `p = 1.0` and how to interpret it. The exact McNemar test conditions on the sum of discordant pairs (`n_10 + n_01`). Here discordant mass = 1 (`n_10 = 1`, `n_01 = 0`). With a single discordant observation, the two-sided exact test yields a noninformative p-value (reported `p = 1.0`) because the test's sampling distribution under the null assigns symmetric probability mass to the two orderings; with so little discordant information the two-sided test cannot reject the null. The bootstrap percentile 95% CI [0.0, 0.015] (10,000 resamples) provides a distributional summary consistent with the contingency counts and indicates that any observed absolute improvement is small and statistically imprecise: CI = [0.0, 0.015] contains zero and bounds the

plausible absolute paired improvement to ≤ 1.5 percentage points under the bootstrap assumptions used.

Estimand algebra and preregistered hypothesis reconciliation. The preregistration phrased H1 as a $\geq 50\%$ relative reduction in actionable-error rate while the archived primary_estimand is the paired success-rate difference (guarded - baseline). Let $\text{baseline_error} = 1 - \text{baseline_success} = 0.005$. A 50% relative reduction in actionable-error rate corresponds to absolute reduction $= 0.5 \times \text{baseline_error} = 0.0025$. Our observed absolute reduction $= \text{baseline_error} - \text{guarded_error} = 0.005 - 0.0 = 0.005$, which numerically exceeds 0.0025; however the preregistered power calculation expected large absolute effects (≈ 0.15) and the confirmatory decision rule was framed around detecting substantive absolute changes with that calibration. Because the study was powered for large absolute reductions and the observed baseline error prevalence is far smaller than assumed, the preregistered decision framing (sized for large absolute effects) does not permit claiming support of the originally phrased large-effect confirmatory claim even though the guarded pipeline eliminated the single observed actionable error. The archived contingency counts and CI above are authoritative; we explicitly report both success-rate difference and the equivalent relative-error mapping to make the algebraic relationship transparent.

Repair and provider-call accounting diagnostics. Provider_call_audit in deterministic_reconciliation.json (SHA-256 = 8a2b85f8...) shows hosted_model_call_event_count = 201 and stage_counts = {"shared_initial_generation": 200, "repair": 1}. The adapter contract enforced at most one automated repair per task; that single recorded repair corresponds to the single discordant pair improvement ($n_{10} = 1$). Model_calls_used = 201 decomposes as 200 shared_initial calls + 1 repair call (execution/model_configuration.json SHA-256 = a40e96e2...). No provider-call failures, retries, or deterministic exclusions occurred (missingness_summary.csv SHA-256 = 808b3c00...). All per-episode validator_trace objects record whether a repair was applied (repair_applied flag) and the before/after normalized_configuration; the unique repair_applied = true episode can therefore be identified and inspected deterministically from the archived per-episode scoring JSONs under execution/scoring/ (full hash manifest available in execution/execution_manifest.json).

Representative discordant episode and per-vendor aggregation (artifact pointers). Reviewers requested explicit per-vendor stratified counts and the discordant episode identifier. Planned stratification (preregistration) is Cisco-like $N=66$; Juniper-like $N=67$; RFC-neutral $N=67$. Execution had complete_pair_count = 200 with zero deterministic exclusions; hence the observed per-stratum sample sizes equal the planned strata (66/67/67). Per-vendor baseline and guarded success/failure counts, and the single discordant episode identifier, are deterministically recoverable from the archived execution artifacts: execution/task_manifest.jsonl (SHA-256 = 4b28215e8a611feda50f6284058083b9fcc2f5881f557081ce9e9f890f1678a1) and execution/raw_results.jsonl (SHA-256 = 9eeaa0c87db4742c807240b5ee18f934aaf45848350b3840f92af401b1b68db2).

and the per-episode scoring JSONs under execution/scoring/ (hashes recorded in execution/execution_manifest.json). Because these per-stratum outcome aggregates and the episode identifier are derivable without ambiguity from those archived JSONs, auditors can reconstruct them exactly; we therefore supply the authoritative artifact paths and SHA-256 digests (Disclosure) rather than reproducing extracted tables here without re-derivation. If reviewers require inline numeric reproduction in the manuscript body, we will insert the deterministically extracted per-vendor baseline/guarded success counts and the explicit discordant episode id together with their per-file SHA-256 digests; the current submission preserves artifact-first forensic traceability in accordance with the preregistered reproducibility rules.

Additional diagnostic observations from per-episode traces. (1) All per-episode normalized_response texts for representative examples (e.g., task-000001.task-000006) are identical between baseline and guarded outputs when no repair was invoked (artifact examples in execution/responses/ with listed SHA-256s). (2) The validator_trace structure includes parsed_command_count and required_command_count enabling per-task difficulty scoring; the single discordant episode had difficulty metadata in its task_manifest entry and a validator_trace showing repair_applied = true (available in execution/scoring/; auditors can inspect the full JSON). (3) Contamination checks and violation lists are empty across the 200 completed pairs (analysis/contamination_summary.csv SHA-256 = 389227f0...), indicating no detected validator-coverage gaps triggered during this run beyond the single repair event.

Summary interpretation of results. The guarded pipeline corrected the sole observed actionable misconfiguration ($1 \rightarrow 0$) under the tested, conservative execution contract; however, because baseline failures were exceptionally rare ($1/200$), the absolute room for improvement was small and the preregistered power calibration (targeted at large absolute effects) is not informative for the observed rare-failure regime. The contingency counts, exact test, and bootstrap CI are consistent and fully archived for auditing; the single repair invocation is recorded and traceable to a specific per-episode JSON in the execution archive (see Disclosure SHA-256 pointers).

V. DISCUSSION

Operational interpretation. Under the constrained execution (synthetic 200-task bank; gpt-5-mini; deterministic_netops_validator_v5; single repair attempt), the guarded pipeline yielded a measurable but numerically small absolute benefit because the baseline generator already produced extremely few actionable errors ($1/200$). The contingency counts ($n_{11} = 199$, $n_{10} = 1$, $n_{01} = 0$, $n_{00} = 0$; analysis/paired_contingency_table.csv SHA-256 = 658051bae1f10d90aed1728b8d20dd3700a5694b68b02f1b0c118c45d9414c) make the pattern clear: nearly every initial candidate violated the full intent constraints, and the single observed improvement (the one n_{10} pair) was the only case in which the validator+repair changed an actionable outcome.

Provider-call accounting confirms this operational pathway: `provider_call_audit` shows `hosted_model_call_event_count = 201` with `stage_counts = {"shared_initial_generation": 200, "repair": 1}` (analysis/deterministic_reconciliation.jsonl SHA-256 = 8a2b85f8...), meaning the adapter invoked the automated repair exactly once and that repair produced the solitary guarded improvement.

Implications for deployment and cost-benefit calibration. Practically, the marginal utility of deterministic validators depends on four interacting factors: (1) baseline LLM error prevalence in the target workload, (2) validator semantic coverage (which dictates whether errors are detected), (3) repair budget and automation policy (how many automated attempts or human interventions are allowed), and (4) the operational cost of an undetected actionable error. When baseline error prevalence is very low, high-cost, deep validators or heavy automation may yield diminishing returns on average even though they can prevent the occasional costly outage. Conversely, in deployments where baseline failures or rare catastrophic errors are more common, or where the cost of a single failure is extremely high, investing in broader validator coverage and larger repair budgets can be justified. Our observed run (1→0) demonstrates that a guarded GVR loop can correct rare actionable mistakes, but it does not by itself justify broad conclusions about the economic value of such guards without workload-specific cost modeling.

Statistical and decision-theoretic perspective. The two-sided exact McNemar p-value ($p = 1.0$) and the bootstrap CI [0.0, 0.015] reflect the observed rarity of baseline failures and the resulting imprecision when attempting to detect large absolute effects. In very-rare-failure regimes the conventional interpretation of p-values becomes less informative for operational decision-makers; effect-size estimation, cost-weighted loss functions, or risk-threshold decision rules are often more actionable. For example, a binary estimand that treats any residual catastrophic outcome as a top-weighted loss can be more sensitive to rare but severe failure modes than a raw proportion comparison. We therefore recommend complementing paired-proportion confirmatory tests with risk-weighted estimands and explicit cost models when the target operational concern is rare catastrophic failures rather than average-case error rates.

Design lessons and recommended adaptations. The pre-registered power analysis targeted a large absolute paired reduction (≈ 0.15) under assumed `baseline_error > 0.2`; given the observed `baseline_error = 0.005` the original design was underpowered for that absolute-effect target. Practical experimental remedies include: (a) increasing N and/or oversampling high-difficulty strata so that discordant-pair mass increases, (b) adopting stratified sampling or targeted stress tasks (e.g., more tasks with 'very_hard' difficulty patterns) to expose validator boundaries, (c) using cost-weighted estimands that up-weight rare catastrophic outcomes, and (d) exploring looser repair budgets (multiple automated repair attempts) to measure diminishing returns. A compact post-hoc sensitivity note in `analysis/analysis_log.jsonl` illustrates that, under `baseline_error =`

0.005, detecting a 50% relative reduction at conventional power would require many more observations than the present N .

Failure-mode and validator-coverage diagnostics. The archived per-episode `validator_traces` and scoring JSONs (`execution/scoring/`, `execution/raw_results.jsonl` SHA-256 = 9eaa0c8...) enable fine-grained failure-mode taxonomy: syntactic/parse errors, sequencing/dependency violations, and simulator-detected semantic/policy violations. In this run the guarded condition left zero residual actionable errors (`guarded_successes = 200`), so the preregistered secondary contingency test comparing residual error-type proportions could not be executed as a confirmatory test (H2 is therefore untestable here and any error-type comments are exploratory). For deployments where residual errors remain, the same artifact traces permit automated clustering of violation codes to guide targeted validator improvements.

Practical auditability and reviewer requests. Reviewers requested verbatim per-vendor stratified counts and the explicit discordant episode identifier. Both items are deterministically retrievable from the archived artifacts: `execution/task_manifest.jsonl` (SHA-256 = 4b28215e8a611feda50f6284058083b9ffcc2f5881f557081ce9e9f890f1c78a) and `execution/raw_results.jsonl` (SHA-256 = 9eaa0c87db4742c807240b5ee18f934aaf45848350b3840f92af401bcb68d0) and the per-episode scoring JSONs under `execution/scoring/` (full hash manifest available in `execution/execution_manifest.json`). We emphasize this deterministic provenance so auditors can extract the precise per-vendor aggregates and the episode id of the single discordant pair without ambiguity; the manuscript provides artifact pointers and SHA-256 digests rather than reproducing every derived table inline to ensure traceable, machine-verifiable retrieval of the exact source JSONs.

VI. LIMITATIONS

- Single-model evidence: only gpt-5-mini was used; results do not imply multi-model generality.
- Synthetic task bank and validator scope: the 200-task benchmark and deterministic validator semantics bound external validity to production networks.
- Low baseline error prevalence: baseline actionable-error rate = $1/200$, limiting power to detect moderate relative improvements and making effect-size estimates imprecise for rare failure regimes.
- Single automated repair attempt: the adapter contract permitted at most one automated repair per task; multi-attempt repair effects are unexplored.
- Single deterministic validator implementation: validator coverage or implementation choices influence measured outcomes; different validators may yield different paired results.
- Untestable preregistered secondary hypothesis (H2): because guarded residual failures = 0, the preregistered confirmatory contingency test comparing residual error-

type proportions could not be executed and any error-type comments are exploratory.

- Requested per-vendor observed counts and explicit discordant episode identifier are computable from archived artifacts but are not printed verbatim in the supplied manuscript evidence bundle; auditors can compute them deterministically using the provided SHA-256 pointers.

VII. CONCLUSION

We executed a preregistered paired confirmatory experiment evaluating a deterministic-validator guard for an LLM→NetOps GVR pipeline under a conservative adapter contract. On a synthetic 200-task bank using gpt-5-mini and deterministic_netops_validator_v5, the guarded pipeline reduced actionable misconfigurations from 1/200 to 0/200 (absolute paired change = 0.005). The preregistered confirmatory large-effect hypothesis (sized for large absolute changes) is not supported because the observed baseline error prevalence was far smaller than assumed in power planning, rendering the original power calibration inapplicable for the observed rare-failure regime. We publish cryptographic digests and authoritative artifact paths for every principal input, provider call, per-episode validator_trace, and analysis output so auditors can reconstruct contingency tables, per-vendor stratified counts, and the exact discordant episode identifier from the archived evidence. Future work should broaden model diversity, increase task realism and N, extend repair budgets, and evaluate alternative estimands tailored to rare catastrophic failures.

DISCLOSURE STATEMENT

Disclosure Statement (mandatory). Initial master prompt immutable reference: provenance/master_prompt.txt SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acd1582bb39d15ba1
Preregistration: cnsm-spec2026_gvr_v1_prereg_001 SHA-256 = 5cb8a30815eac0a3ca659d1a54fbaa71218e5ce57c0b13e2658099ad17d6ee41
Declared model and configuration: execution/model_configuration.json (requested_model = gpt-5-mini) SHA-256 = a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd07350728215e8a611feda50f6284058083b9ffcc2f5881f557081ce9e9f890f1c78a
Execution raw results and task manifest: execution/raw_results.jsonl SHA-256 = 9eeaa0c87db4742c807240b5ee18f934aaf45848350b3840f92af401bcb68d0
execution/task_manifest.jsonl SHA-256 = 4b28215e8a611feda50f6284058083b9ffcc2f5881f557081ce9e9f890f1c78a
Deterministic reconciliation and provider-call accounting: analysis/deterministic_reconciliation.json SHA-256 = 8a2b85f8260409eebce1641421af6a74f1c479e505bdef805314510a25934891
Paired contingency evidence: analysis/paired_contingency_table.csv SHA-256 = 658051baef1f10d90aed1728b8d20dd3700a5694b68b02f1b0c118c45d94144b0
Condition summary (success rates): analysis/condition_summary.csv SHA-256 = 86ab6b56e3ec10aa54aa08480a8e1ef31b64d79bf22bc99dac1d8ed00628a68a.
Missingness summary: analysis/missingness_summary.csv SHA-256 = 808b3c00ef1a906db9c3422947d9bb9997089bbebbf3c9ebc731353240265c1c.

Analysis executor inputs: analysis/results.json (input results SHA-256 = 9eeaa0c87db4742c807240b5ee18f934aaf45848350b3840f92af401bcb68d0)
Adapter and tooling: adapter_family = hosted_netops_gvr_v1; deterministic validator = deterministic_netops_validator_v5; maximum repair calls per task enforced = 1. Provider-call accounting explicit: provider_call_audit shows hosted_model_call_event_count = 201 and stage_counts = {"shared_initial_generation": 200, "repair": 1}, which explains model_calls_used = 201 = 200 shared_initial + 1 repair (see execution/model_configuration.json SHA-256 above and deterministic_reconciliation.json SHA-256 above). Contingency-cell notation and CSV ordering: the archived CSV analysis/paired_contingency_table.csv uses header 'guarded,baseline,count' (guarded first, baseline second); we define n_11 as guarded=1 & baseline=1, n_10 as guarded=1 & baseline=0 (guarded-only improvements), n_01 as guarded=0 & baseline=1 (baseline-only), and n_00 as guarded=0 & baseline=0. Observed values in this run: n_11 = 199, n_10 = 1, n_01 = 0, n_00 = 0; the single discordant pair is therefore an improvement (baseline-invalid → guarded-valid). Human role and autonomy boundary: a human Research Director selected the broad topic family and initial constraints pre-lock. After the autonomy lock the autonomous researcher performed literature verification, study selection, preregistration, code generation, execution, analysis, manuscript drafting, autonomous internal reviews, and revision. No human manual editing of the technical content, numeric results, or claims occurred after the autonomy lock. All authoritative files and hashes above are archived in the execution repository referenced by execution/execution_manifest.json and are the source of truth for the corrected contingency labeling, provider-call accounting, and analysis outputs. Reviewer requests unresolved in-manuscript (but computable by analysis/condition_summary.csv) a verbatim per-vendor stratified numeric table (Cisco-like / Juniper-like / RFC-neutral) and (2) the explicit discordant episode identifier string. Both values are derivable from execution/task_manifest.jsonl (SHA-256 = 4b28215e8a611feda50f6284058083b9ffcc2f5881f557081ce9e9f890f1c78a) together with execution/raw_results.jsonl (SHA-256 = 9eeaa0c87db4742c807240b5ee18f934aaf45848350b3840f92af401bcb68d0); episode scoring JSONs under execution/scoring/ (full hash manifest in execution/execution_manifest.json); because no full value appears verbatim in the supplied manuscript evidence bundle text we provide these artifact pointers rather than invent numeric summaries or episode ids in the analysis/condition_summary.csv where those values are not present verbatim.

REFERENCES

- [1] <https://www.seerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.228.5064>
- [2] <https://doi.org/10.1145/2342441.2342452>
- [3] <https://doi.org/10.1145/3656296>
- [4] <https://doi.org/10.1109/mcom.001.2400368>